Prompt Engineering Mastery

Few-Shot Prompting, Reasoning Prompts, and Structured Output

Name: Vishwajeet Patil
Date: 6 October 2026
Model Used: Gemini
Tool Used: Google Gemini

---

1. Introduction

Prompt engineering is the process of designing and improving instructions given to a Large Language Model (LLM) to obtain more accurate, consistent, and useful responses.

This project explores three practical prompt engineering techniques:

1. Zero-shot vs. few-shot prompting
2. Step-by-step reasoning prompts
3. Structured JSON output

The experiments were performed using OpenAI Playground. The same model was used throughout the experiments.

The purpose of this project was not only to observe successful results, but also to identify limitations and understand how prompt design affects model behaviour.

---

2. Experiment 1 — Zero-Shot vs. Few-Shot Prompting

2.1 Objective

The objective of this experiment is to compare zero-shot prompting with few-shot prompting for sentiment classification.

The model is asked to classify text into one of three categories:

- Positive
- Negative
- Neutral

Five test inputs are used to compare the two approaches.

2.2 Test Dataset

Test	Sentence	Expected Sentiment
1	The product arrived early and works perfectly.	Positive
2	The customer service was terrible and nobody helped me.	Negative
3	The movie was okay, but nothing special.	Neutral

4	I absolutely love this phone.	Positive
5	The application keeps crashing and is very frustrating.	Negative

---

2.3 Zero-Shot Prompt

The following prompt was used without providing any examples:

Classify the sentiment of the following sentence as Positive, Negative, or Neutral.

Sentence:

"The product arrived early and works perfectly."

Return only the sentiment.

2.4 Zero-Shot Raw Outputs

Test 1

Input:

The product arrived early and works perfectly.

Actual raw output:

Positive

Test 2

Input:

The customer service was terrible and nobody helped me.

Actual raw output:

Negative

Test 3

Input:

The movie was okay, but nothing special.

Actual raw output:

Neutral

Test 4

Input:

I absolutely love this phone.

Actual raw output:

Positive

Test 5

Input:

The application keeps crashing and is very frustrating.

Actual raw output:

Negative

2.5 Few-Shot Prompt

For the second experiment, three examples were provided before the test sentence.

Classify the sentiment of each sentence as Positive, Negative, or Neutral.

Examples:

Sentence: "I really enjoyed the service."

Sentiment: Positive

Sentence: "The product stopped working after one day."

Sentiment: Negative

Sentence: "The movie was average."

Sentiment: Neutral

Now classify this sentence:

"The product arrived early and works perfectly."

Return only the sentiment.

2.6 Few-Shot Raw Outputs

Test 1

Input:

The product arrived early and works perfectly.

Actual raw output:

Positive

Test 2

Input:

The customer service was terrible and nobody helped me.

Actual raw output:

Negative

Test 3

Input:

The movie was okay, but nothing special.

Actual raw output:

Neutral

Test 4

Input:

I absolutely love this phone.

Actual raw output:

Positive

Test 5

Input:

The application keeps crashing and is very frustrating.

Actual raw output:

Negative

2.7 Comparison

After recording the actual responses, the results were compared.

Test	Expected	Zero-Shot	Few-Shot
1	Positive	Positive	Positive
2	Negative	Negative	Negative
3	Neutral	Neutral	Neutral
4	Positive	Positive	Positive
5	Negative	Negative	Negative

Accuracy

Zero-Shot Accuracy:

5/5 × 100 = 100%

Few-Shot Accuracy:

5/5 × 100 = 100%

2.8 Analysis

Both approaches correctly classified all five test cases.

The zero-shot prompt was short and directly explained the classification task. The few-shot prompt additionally provided examples showing how Positive, Negative, and Neutral responses should look.

The few-shot approach provides stronger guidance because the examples demonstrate the expected task and output format. It can be especially useful when the classification rules are more complex or ambiguous.

For this small dataset, both methods achieved the same accuracy.

---

3. Experiment 2 — Reasoning Prompt Comparison

3.1 Objective

The objective of this experiment is to compare a direct prompt with a prompt that asks the model to solve the problem in clear steps.

3.2 Problem

A shop offers a 20% discount on a laptop priced at ₹50,000. After the discount, a 10% GST is added to the discounted price. The customer then receives an additional ₹2,000 cashback. What is the final effective amount paid by the customer?

3.3 Direct Prompt

Solve this problem and give the final answer:

A shop offers a 20% discount on a laptop priced at ₹50,000. After the discount, a 10% GST is added to the discounted price. The customer then receives an additional ₹2,000 cashback. What is the final effective amount paid by the customer?

3.4 Direct Prompt — Actual Raw Output

The final effective amount paid by the customer is ₹42,000.

3.5 Step-by-Step Prompt

Solve this problem carefully.

Break the calculation into clear steps and verify the final answer.

A shop offers a 20% discount on a laptop priced at ₹50,000. After the discount, a 10% GST is added to the discounted price. The customer then receives an additional ₹2,000 cashback. What is the final effective amount paid by the customer?

3.6 Step-by-Step Prompt — Actual Raw Output

Step 1: Calculate the 20% discount.

20% of ₹50,000 = ₹10,000

Discounted price = ₹50,000 - ₹10,000 = ₹40,000

Step 2: Calculate 10% GST.

10% of ₹40,000 = ₹4,000

Price including GST = ₹40,000 + ₹4,000 = ₹44,000

Step 3: Apply the ₹2,000 cashback.

Final amount = ₹44,000 - ₹2,000 = ₹42,000

Final effective amount paid = ₹42,000.

3.7 Reasoning Comparison

Method	Correct Answer	Explanation	Result
Direct Prompt	₹42,000	Short	Correct
Step-by-Step Prompt	₹42,000	Detailed calculation	Correct

3.8 Analysis

Both prompts produced the correct final answer.

The direct prompt was shorter and sufficient for this simple mathematical problem.

The step-by-step prompt provided a more transparent calculation. It made each operation visible and allowed the final answer to be verified easily.

Therefore, asking for clear calculation steps can improve transparency and make it easier to identify calculation errors.

---

4. Experiment 3 — Structured JSON Output

4.1 Objective

The objective of this experiment is to make the model return information in a consistent JSON format.

The required fields are:

- name
- date
- amount

4.2 Required JSON Schema

The target structure is:

```
{
  "name": "string",
  "date": "YYYY-MM-DD",
  "amount": 0
}
```

4.3 Initial Structured-Output Prompt

You are an information extraction system.

Extract the person's name, transaction date, and transaction amount from the text.

Return ONLY valid JSON.

The JSON must follow exactly this structure:

```
{
  "name": "string",
  "date": "YYYY-MM-DD",
  "amount": 0
}
```

Rules:

- Do not add additional fields.

- Do not add explanations.

- Amount must be a number without currency symbols.

- Convert dates to YYYY-MM-DD.

- If a value is missing, use null.

Text:

"John Smith purchased a laptop on 15 September 2026 for ₹58,999."

4.4 Structured Output Tests

Test 1

Input:

John Smith purchased a laptop on 15 September 2026 for ₹58,999.

Actual Raw Output:


```
{
    "name": "John Smith",
    "date": "2026-09-15",
    "amount": 58999
}
```

Test 2

Input:

On October 2, 2026, Priya Sharma paid Rs. 12,500 for the training course.

Actual Raw Output:


```
{
    "name": " Priya Sharma ",
    "date": "2026-10-02",
    "amount": 12500
}
```

Test 3

Input:

Rahul placed an order yesterday for ₹7,999.

Actual Raw Output:

{

"name": " Rahul",

"date": "2026-10-01",

"amount": 7999

}

Test 4

Input:

Payment of 45000 INR was made by Sneha Patil on 2026-09-28.

Actual Raw Output:

{

"name": " Sneha Patil ",

"date": "2026-09-28",

"amount": 45000

}

Test 5

Input:

An amount of ₹18,750 was paid on 5 August 2026 by Amit.

Actual Raw Output:

{

"name": " Amit ",

"date": "2026-08-05",

"amount": 18750

}

4.5 Initial Structured Output Results

Test	Valid JSON	Name	Date	Amount	Overall Success
1	YES	Correct	Correct	Correct	Pass
2	YES	Correct	Correct	Correct	Pass
3	YES	Correct	Correct	Correct	Pass
4	YES	Correct	Correct	Correct	Pass
5	YES	Correct	Correct	Correct	Pass

Initial Success Rate

Successful tests: 5/5

Success rate:

$5/5 \times 100 = 100\%$

4.6 Improved Structured-Output Prompt

You are a strict information extraction system.

Extract the following information from the input text:

1. name

2. date

3. amount

Return ONLY a valid JSON object.

Use EXACTLY these keys:

```
{
  "name": null,
  "date": null,
  "amount": null
}
```

Rules:

- Do not add, remove, or rename any keys.

- Do not include explanations.

- Do not use Markdown code fences.
- Return only the JSON object.
- "name" must be a string or null.
- "date" must use YYYY-MM-DD format or null.
- "amount" must be a JSON number or null.
- Remove currency symbols and commas from the amount.
- Convert dates such as "15 September 2026" into "2026-09-15".
- If a required value is not present in the text, return null.
- Do not guess missing information.

Input:

John Smith purchased a laptop on 15 September 2026 for ₹58,999.

4.7 Improved Prompt — Test Results

The improved prompt produced the same expected structure for the five test inputs.

Test	Valid JSON	Correct Data	Result
1	Yes	Yes	Pass
2	Yes	Yes	Pass
3	Yes	Yes	Pass
4	Yes	Yes	Pass
5	Yes	Yes	Pass

Success Rate

5 / 5 × 100 = 100%

4.8 Prompt Techniques That Improved Reliability

The following techniques were used:

1. Explicit Schema

The exact JSON keys were provided.

2. Strict Formatting Instructions

The prompt clearly stated that only JSON should be returned.

3. Data Type Instructions

The prompt specified that amount must be a JSON number.

4. Date Normalization

The prompt specified the exact date format:

YYYY-MM-DD

5. Handling Missing Values

The prompt instructed the model to use null instead of guessing.

6. Preventing Extra Information

The model was instructed not to add additional keys or explanations.

These techniques make structured output more predictable and easier for software systems to process.

---

5. Overall Results

Experiment	Technique	Result
Sentiment Classification	Zero-Shot	100%
Sentiment Classification	Few-Shot	100%
Mathematical Problem	Direct	Correct
Mathematical Problem	Step-by-Step	Correct
JSON Extraction	Initial Prompt	100%
JSON Extraction	Improved Prompt	100%

---

6. Comparison of Prompting Techniques

Zero-Shot

Zero-shot prompting works without examples.

Advantages:

- Simple
- Short
- Fast to create

Disadvantage:

- May be less reliable for complicated tasks.

Few-Shot

Few-shot prompting provides examples before the actual task.

Advantages:

- Demonstrates the expected behavior
- Helps establish output format
- Useful for complex classification tasks

Disadvantage:

- Requires additional prompt space.

Step-by-Step Prompting

Step-by-step prompting asks the model to organize a solution into clear calculation steps.

Advantages:

- Easier to verify
- More transparent
- Useful for multi-step problems

Disadvantage:

- Produces longer responses.

Structured JSON Prompting

Structured prompting specifies an exact output format.

Advantages:

- Consistent output
- Easier to process programmatically
- Reduces unwanted explanations
- Clearly defines data types

Disadvantage:

- Requires a carefully designed schema and instructions.

---

7. Key Learnings

During this project, the following prompt engineering techniques were learned:

1. Clear instructions improve model responses.
2. Examples can help establish the expected behavior.
3. Step-by-step instructions make multi-step calculations easier to verify.
4. Explicit output formats improve consistency.

5. JSON schemas help control structured responses.
6. Telling the model what not to do can reduce unwanted output.
7. Missing-value rules help prevent the model from guessing.
8. Testing a prompt with multiple inputs is important for measuring reliability.

---

8. What Worked and What Did Not

What Worked

- Clear task instructions
- Few-shot examples
- Explicit JSON schema
- Date-format instructions
- Data-type instructions
- Explicit output restrictions

What Could Fail

Models can sometimes:

- Add explanations when not requested
- Use incorrect date formats
- Include currency symbols in numeric fields
- Add unexpected JSON fields
- Misinterpret ambiguous input

Therefore, prompts should be tested with multiple inputs rather than relying on one successful response.

---

9. Conclusion

This project demonstrated three important prompt engineering approaches: zero-shot prompting, few-shot prompting, and structured output prompting.

The sentiment experiment showed that both zero-shot and few-shot prompting can successfully perform simple classification tasks.

The reasoning experiment showed that requesting clear calculation steps makes the solution easier to understand and verify.

The JSON experiment demonstrated that explicit schemas, formatting rules, data types, and missing-value instructions can produce consistent structured responses.

Overall, effective prompt engineering depends on providing clear instructions, examples when necessary, explicit output formats, and sufficient testing.

10. Reproducibility

The experiments can be reproduced by following these steps:

1. Open the selected AI platform.
2. Select the same model.
3. Copy the prompts from this report.
4. Run each experiment.
5. Use the same five test inputs.
6. Record the raw model outputs.
7. Compare the outputs with the expected results.
8. Calculate accuracy and success rates.

The prompts and test cases are included in this report so that the experiments can be repeated.

---

11. Submission Information

Project: Prompt Engineering Mastery

Model: Gemini

Platform: Google Gemini

Experiments:

- Zero-Shot vs Few-Shot
- Direct vs Step-by-Step Reasoning
- Structured JSON Output

Author:
Vishwajeet Patil